FIREBRAND
LYCEUM EDUCATION GROUP

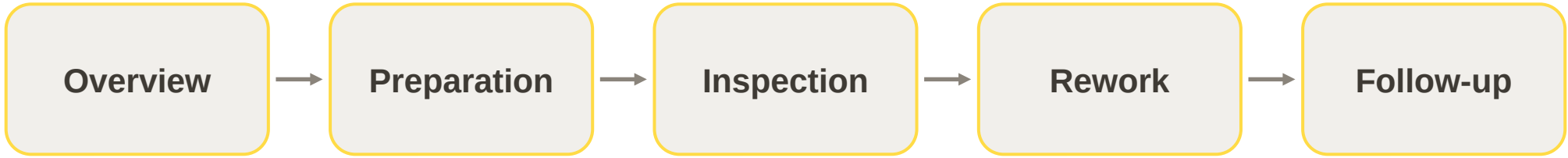
Module 4: Software Design
Session 6 stretch: Reviews, feedback and reflection

Software Developer Level 4



Fagan inspections: reviewing as a process

Michael Fagan, IBM, 1976: a small team with defined roles inspects a design in five operations.



Overview	Preparation	Inspection	Rework	Follow-up
The designer walks the team through the design	Each person studies it alone	The team finds errors; nobody redesigns	The designer fixes every error found	The moderator checks that each fix is made

Why it matters: keeping 'find' apart from 'fix' keeps a review short and stops it becoming a design meeting.

Try this: which operation did your review board leave out today, and what did that cost?

FIREBRAND LYCEUM EDUCATION GROUP

Schön: the reflective practitioner

The idea

Professionals know more than they can put into words, and they learn by reflecting on what they do.

Reflection-in-action: adjusting while you work.
Reflection-on-action: looking back afterwards.

Why it matters for a developer

Schön described design as a reflective conversation with the situation: you sketch, the sketch 'talks back', you adjust.

Writing the reflection down turns a one-off fix into a habit.

Try this: when did your design 'talk back' to you today, and what did you change?

FIREBRAND
LYCEUM EDUCATION GROUP

Psychological safety

EDMONDSON, 1999

> **Teams learn when it is safe to admit a mistake or ask a question.**

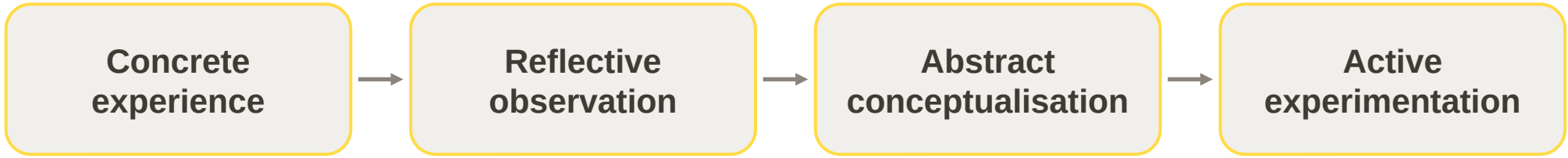
Edmondson defined it as a shared belief that the team is safe for interpersonal risk taking; in 51 work teams it went with more learning behaviour.

Try this: what would make it safe for the newest developer in your team to say 'I think this design is wrong'?

FIREBRAND
LYCEUM EDUCATION GROUP

Kolb's experiential learning cycle

David Kolb (1984): we learn by going round a cycle, not by hearing a rule once.



Concrete experience	Reflective observation	Abstract conceptualisation	Active experimentation
You do something: today, the review board	You look back at what happened, and why	You form a rule or idea you can reuse	You try it on the next design, and go round again

Contested: the 'learning styles' built on this cycle, and matching teaching to them, lack good evidence.

Try this: where does your Activity 3 goal sit on the cycle, and what is the next stage?

FIREBRAND
LYCEUM EDUCATION GROUP

The ladder of feedback

David Perkins (Project Zero): climb four rungs, in order, when you give feedback.

Rung	What you do	On a design review
1 Clarify	Ask questions to understand fully	'Who uses this screen first?'
2 Value	Say what you like, specifically, and why	'The error keeps what was typed: keep that.'
3 State concerns	Name worries kindly: 'I wonder if...'	'I wonder if contractors see too much.'
4 Suggest	Offer improvements: 'What if...'	'What if the rule lived in one class?'

> **Try this: rewrite one of today's feedback points so that it climbs all four rungs.**

FIREBRAND LYCEUM EDUCATION GROUP

Gibbs' reflective cycle

Graham Gibbs (1988): six stages that turn an experience into a plan.

1. **Description**
   What happened?
2. **Feelings**
   What were you thinking and feeling?
3. **Evaluation**
   What was good and bad about it?
4. **Analysis**
   What sense can you make of it?
5. **Conclusion**
   What else could you have done?
6. **Action plan**
   If it happened again, what would you do?

Try this: take your review board turn through all six stages in your Reflective Log.

FIREBRAND
LYCEUM EDUCATION GROUP

FIREBRAND
LYCEUM EDUCATION GROUP

Module 4: Software Design
Session 6 stretch: Reviews, feedback and reflection

Software Developer Level 4

Optional self-study after Session 6. Six short ideas behind today's session, one on each slide: Fagan inspections, Schön's reflective practitioner, psychological safety, Kolb's experiential learning cycle (with a contested part flagged), the ladder of feedback and Gibbs' reflective cycle.

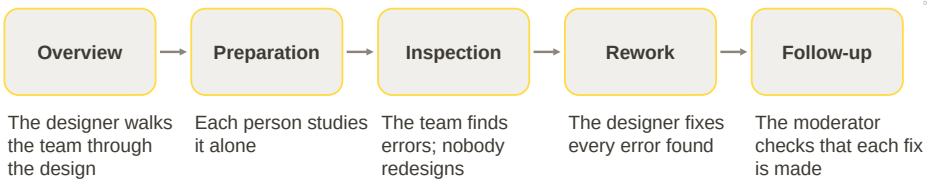
How to use it: read one slide at a time, then answer its "Try this" question in two or three sentences, about Kestrel Moor or your own project. Your answers can go in your Reflective Log or your OneFile reflection; none of this is assessed.

Each slide's notes give the idea in more detail, why it matters for a developer, an answer guide for the "Try this" question, and the full reference with the month it was checked.

Keep it confidential: no personal or customer data, credentials or confidential material in your answers, and no employer data in AI tools your employer has not approved.

Fagan inspections: reviewing as a process

Michael Fagan, IBM, 1976: a small team with defined roles inspects a design in five operations.



Overview	Preparation	Inspection	Rework	Follow-up
The designer walks the team through the design	Each person studies it alone	The team finds errors; nobody redesigns	The designer fixes every error found	The moderator checks that each fix is made

Why it matters: keeping 'find' apart from 'fix' keeps a review short and stops it becoming a design meeting.

Try this: which operation did your review board leave out today, and what did that cost?

FIREBRAND LYCEUM EDUCATION GROUP

2

The idea: Michael Fagan formalised design and code inspections at IBM. A team of about four people with defined roles (moderator, designer, coder or implementer, and tester) works through five operations: overview, preparation, inspection, rework and follow-up. The inspection meeting only finds errors: it does not redesign or hunt for solutions. The moderator reports the findings within a day and makes sure every error is resolved.

Why it matters for a developer: a review that tries to find and fix at once runs long and drifts into redesign. Fagan also insisted that inspection results must never be used to appraise programmers, so people keep reporting errors honestly.

Answer guide: most boards skip individual preparation (reading the pack alone before the meeting) and formal follow-up (someone checking the changes were made). The cost is questions spent on understanding rather than finding problems, and no guarantee that feedback is acted on. Activity 3 was a light follow-up.

Caution: the paper also says errors cost 10 to 100 times more to fix later in the process. Treat such multipliers carefully; later researchers dispute how far they generalise.

Reference: Fagan, M. E. (1976) 'Design and code inspections to reduce errors in program development', IBM Systems Journal, 15(3), pp. 182-211. Verified October 2026 against the paper's text.

Schön: the reflective practitioner

The idea

Professionals know more than they can put into words, and they learn by reflecting on what they do.

Reflection-in-action: adjusting while you work. Reflection-on-action: looking back afterwards.

Why it matters for a developer

Schön described design as a reflective conversation with the situation: you sketch, the sketch 'talks back', you adjust.

Writing the reflection down turns a one-off fix into a habit.

Try this: when did your design 'talk back' to you today, and what did you change?

FIREBRAND LYCEUM EDUCATION GROUP

3

The idea: Donald Schön studied how professionals, including engineers, architects and town planners, actually solve problems. He argued that skilled practitioners know more than they can say and rely on judgement learned in practice. He separated reflection-in-action (thinking on your feet and adjusting while you work) from reflection-on-action (looking back afterwards). One chapter treats design as "a reflective conversation with the situation": you make a move, the situation answers back, and you adjust.
Why it matters for a developer: sketching a wireframe or class diagram and noticing that it does not work is exactly this conversation. Writing the reflection down, as in the Reflective Log, is what turns it into something you can reuse.
Answer guide: strong answers name a specific moment and a specific change, for example "When I drew the class diagram, one class was doing three jobs, so I split it and the traceability table got simpler." Weak answers stay general ("I learned a lot about design").
Reference: Schön, D. A. (1983) The Reflective Practitioner: How Professionals Think in Action. New York: Basic Books. Chapter 3 is "Design as a Reflective Conversation with the Situation". Verified October 2026 (publisher listing and contents via Google Books).

Psychological safety

EDMONDSON, 1999

> **Teams learn when it is safe to admit a mistake or ask a question.**

Edmondson defined it as a shared belief that the team is safe for interpersonal risk taking; in 51 work teams it went with more learning behaviour.

Try this: what would make it safe for the newest developer in your team to say 'I think this design is wrong'?

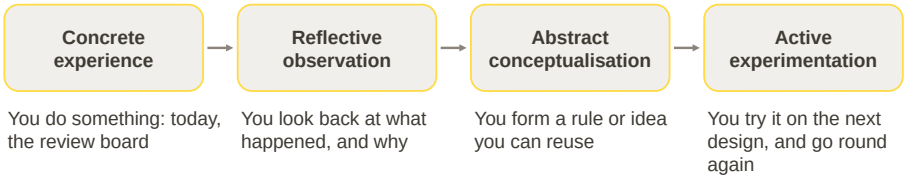
FIREBRAND LYCEUM EDUCATION GROUP

4

The idea: Amy Edmondson introduced team psychological safety as "a shared belief held by members of a team that the team is safe for interpersonal risk taking". In her field study of 51 work teams in a manufacturing company, psychological safety was associated with learning behaviour (asking for help, admitting errors, seeking feedback), and learning behaviour was associated with team performance.
Why it matters for a developer: design reviews only work if people will say "I think this is wrong" and authors will hear it. Psychological safety is not about lowering standards or avoiding criticism; it is about making it safe to speak up.
Answer guide: practical answers include: leads admit their own mistakes in reviews; comments are about the design, never the person; questions are thanked; the most junior person speaks first so they are not anchored by seniors; review findings are never used in appraisals (Fagan's rule, on the previous slide).
Caution: this is one study showing association, not proof that safety alone causes better performance.
Reference: Edmondson, A. (1999) 'Psychological safety and learning behavior in work teams', Administrative Science Quarterly, 44(2), pp. 350-383. Verified October 2026 (JSTOR record and abstract).

Kolb's experiential learning cycle

David Kolb (1984): we learn by going round a cycle, not by hearing a rule once.



You do something: today, the review board

You look back at what happened, and why

You form a rule or idea you can reuse

You try it on the next design, and go round again

Contested: the 'learning styles' built on this cycle, and matching teaching to them, lack good evidence.

Try this: where does your Activity 3 goal sit on the cycle, and what is the next stage?

FIREBRAND LYCEUM EDUCATION GROUP

5

The idea: David Kolb described learning as a cycle of four stages: concrete experience, reflective observation, abstract conceptualisation and active experimentation. You can enter the cycle at any point, but learning sticks when you go all the way round.
Why it matters for a developer: the review board was a concrete experience; your log entry was reflection; turning it into a rule ("always draw the error path") is conceptualisation; using it on your next design is experimentation. The stage most often skipped is reflection.
Contested: Kolb also proposed four learning styles (diverging, assimilating, converging and accommodating) and a questionnaire to measure them. The popular claim that teaching works better when matched to a person's learning style is not supported by good evidence: Pashler and colleagues (2008) found very few studies with a suitable design, and several of those contradicted it. Use the cycle; be sceptical of labels such as "I am a visual learner".
Answer guide: a goal such as "draw the error path for every feature I design" is abstract conceptualisation; the next stage is active experimentation: using it on a named piece of work by a named date, then reflecting again.
References: Kolb, D. A. (1984) Experiential Learning: Experience as the Source of Learning and Development. Englewood Cliffs, NJ: Prentice-Hall. Pashler, H., McDaniel, M., Rohrer, D. and Bjork, R. (2008) 'Learning styles: concepts and evidence', Psychological Science in the Public Interest, 9(3), pp. 105-119. Verified October 2026 (journal page).

The ladder of feedback

David Perkins (Project Zero): climb four rungs, in order, when you give feedback.

Rung	What you do	On a design review
1 Clarify	Ask questions to understand fully	'Who uses this screen first?'
2 Value	Say what you like, specifically, and why	'The error keeps what was typed: keep that.'
3 State concerns	Name worries kindly: 'I wonder if...'	'I wonder if contractors see too much.'
4 Suggest	Offer improvements: 'What if...'	'What if the rule lived in one class?'

Try this: rewrite one of today's feedback points so that it climbs all four rungs.

FIREBRAND LYCEUM EDUCATION GROUP

6

The idea: the ladder of feedback asks reviewers to climb four rungs in order: clarify (ask questions so you understand the work), value (say what you like, specifically, and why), state concerns (kindly, for example "I wonder if..." or "It seems to me..."), and suggest (offer improvements, for example "What if..."). Project Zero at Harvard publishes it as a protocol and credits David Perkins's book.

Why it matters for a developer: many review comments are concerns based on a misunderstanding; clarifying first removes them. Valuing before concerns tells the designer what to keep when they change the design.

Answer guide, on a contractor job view: Clarify: "Does the engineer see the tenant's phone number?" Value: "Showing only the job, access notes and a short handling note is good: it keeps health details out." Concern: "I wonder if free-text access notes could still contain health information." Suggest: "What if the access note had a short character limit and a warning about personal details?"

Note: another Project Zero source says an earlier version of the ladder was developed by Daniel Wilson and colleagues (2005); Perkins's book is the usual citation.

References: Perkins, D. (2003) King Arthur's Round Table: How Collaborative Conversations Create Smart Organizations. John Wiley & Sons. Project Zero, 'The Ladder of Feedback' handout (2019), pz.harvard.edu. Verified October 2026.

Gibbs' reflective cycle

Graham Gibbs (1988): six stages that turn an experience into a plan.

1. **Description**
   What happened?
2. **Feelings**
   What were you thinking and feeling?
3. **Evaluation**
   What was good and bad about it?
4. **Analysis**
   What sense can you make of it?
5. **Conclusion**
   What else could you have done?
6. **Action plan**
   If it happened again, what would you do?

Try this: take your review board turn through all six stages in your Reflective Log.

FIREBRAND LYCEUM EDUCATION GROUP

7

The idea: Graham Gibbs's reflective cycle has six stages: description, feelings, evaluation, analysis, conclusion and action plan. It is widely used in professional training because it stops reflection jumping straight from "what happened" to "what I will do", skipping "why".
Why it matters for a developer: evaluation says what went well or badly; analysis asks why. The "why" is what you carry to the next project.
Answer guide (an example, not a template to copy): Description: I presented to a board of three; the data protection officer asked who reads the comments. Feelings: I was caught out. Evaluation: my technical talk went well; my answer about data did not. Analysis: I designed the screens before thinking about who sees the data. Conclusion: I should trace personal data before drawing screens. Action plan: on my next design at work, list each personal data item and who sees it before wireframing, and ask my lead to check it.
Reference: Gibbs, G. (1988) Learning by Doing: A Guide to Teaching and Learning Methods. Oxford: Further Education Unit, Oxford Polytechnic.